Cloud Platforms (AWS)

Session 2: SQL Fundamentals – Schema Design & CRUD

MIBA24 · Master of Science in Business Analytics

ESADE Business School | February 11, 2026

Oriol Rius Canals | joseporiol.rius@esade.edu

Show Attendance QR Code

Students: scan with your phone to register attendance

Open eCampus

Previous Session Quiz

Quick review of Session 1 concepts

Open Quiz

Session Hook

"If your data isn't structured, your insights will be chaos."

Today we build the foundation for everything that comes next

Today's Agenda

Cloud Storage

Filesystem, Object Storage, Databases

Database Types

SQL vs NoSQL landscape

Relational Design

Schema, Keys, CRUD, AWS RDS

Part 1: Cloud Storage Concepts

Understanding where and how data lives in the cloud

Where Does Your Data Live?

Think about data you've worked with recently...

- Local files?
- Google Drive?
- Database?
- Spreadsheets?

Three Storage Paradigms

How cloud platforms organize data

Filesystem

Traditional hierarchical storage

- ✓ Folders and files
- ✓ Familiar structure
- ✓ Direct access paths

AWS: EFS, FSx

Use: Legacy apps, shared drives

Object Storage

Flat namespace with metadata

- ✓ Unlimited scale
- ✓ Cheap at volume
- ✓ HTTP accessible

AWS: S3 (primary)

Use: Data lakes, backups, ML data

Databases

Structured query interfaces

- ✓ Query languages
- ✓ Transactions/ACID
- ✓ Relationships

AWS: RDS, DynamoDB

Use: Apps, analytics, reporting

This course: We use **S3** for raw data and artifacts, **RDS** for structured analytics queries

Object Storage Deep Dive: Amazon S3

What is S3?

Simple Storage Service — The backbone of AWS data storage

- **Objects** = files (any type, up to 5TB)
- **Buckets** = containers (globally unique names)
- **Keys** = object identifiers (path-like)

Key Concepts

Concept	Example
Bucket	`s3://miba24-analytics/`
Key	`data/sales/2026-01.csv`
Full Path	`s3://miba24-analytics/data/sales/2026-01.csv`

Why S3 for Analytics?

$ **$0.023/GB/month** — Incredibly cheap

∞ **Unlimited scale** — Petabytes, no problem

11 nines durability — 99.999999999%

Native integrations — SageMaker, Glue, Athena

Course Usage

Raw datasets for ML training

Model artifacts from SageMaker

Backups and exports

Filesystem vs Object Storage vs Database

Feature	Filesystem	Object Storage	Database
Structure	Hierarchical (folders)	Flat (buckets/keys)	Tables/Collections
Access	POSIX (mount)	HTTP/API	Query language
Scale	TB	PB+	TB (managed)
Cost	$$$	$	$$
Query	No	Limited (S3 Select)	Full SQL/NoSQL
Transactions	No	No	Yes (ACID)
Best For	Legacy apps, file sharing	Data lakes, backups	Business data, analytics
AWS Service	EFS, FSx	S3	RDS, DynamoDB

Filesystem
"I need a shared drive"

Object Storage
"I need cheap, scalable storage"

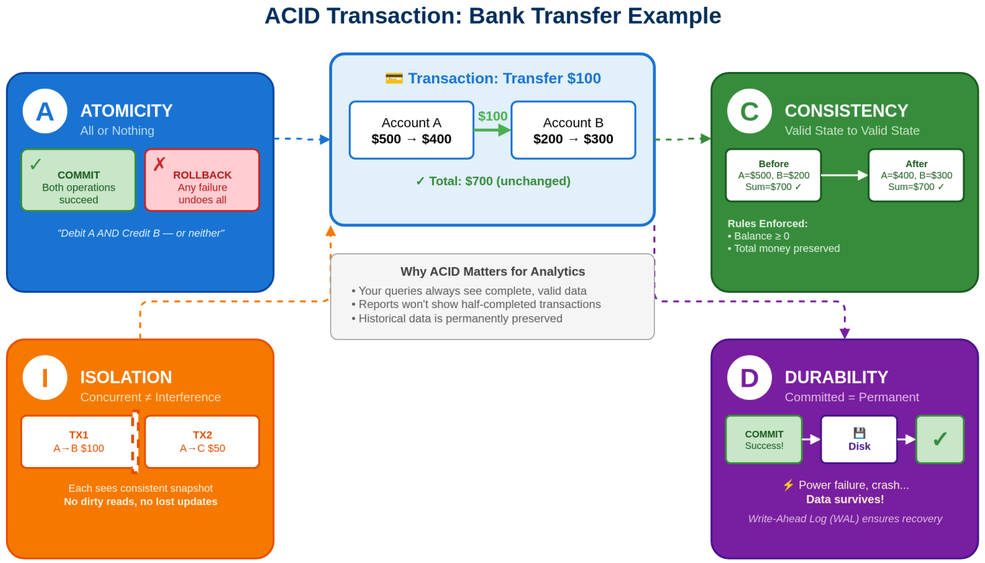
Database
"I need to query and join data"

Part 2: Database Types

SQL vs NoSQL — Understanding the landscape

ACID: Database Reliability Guarantees

The four properties that make transactions trustworthy



💡 **SQL databases guarantee ACID.** This is why banks, airlines, and e-commerce trust them for critical data.

CAP Theorem: Pick Two

The fundamental trade-off in distributed databases (Brewer, 2000)

C

Consistency

Every read returns the most recent write

- ✓ All nodes see same data
- ✓ No stale reads ever
- ✓ Single source of truth

A

Availability

Every request receives a response

- ✓ System never goes down
- ✓ No timeouts or errors
- ✓ Always operational

P

Partition Tolerance

System works despite network failures

- ✓ Survives network splits
- ✓ Nodes can disconnect
- ✓ Distributed by design

⚠️ **Network partitions WILL happen** — so P is mandatory. Your real choice: **CP** or **AP**

CAP: Making the Choice

C + P CP Systems

"I'd rather wait or fail than give you wrong data"

- PostgreSQL
- MongoDB
- Redis Cluster
- RDS

Best for:
Banking, inventory, bookings, analytics

A + P AP Systems

"I'll always respond, even if data is slightly stale"

- Cassandra
- DynamoDB
- CouchDB
- Riak

Best for:
Social feeds, IoT, shopping carts, CDNs

For business analytics: We choose **CP (PostgreSQL/RDS)** — wrong data leads to wrong decisions. Consistency wins.

The Two Database Families

Different tools for different problems

SQL Databases

Relational (SQL)

- ✓ Structured schemas
- ✓ Tables with rows/columns
- ✓ ACID transactions
- ✓ Complex joins
- ✓ Strong consistency

Best for: Structured business data, financial records, reporting

NoSQL Databases

Non-Relational (NoSQL)

- ✓ Flexible schemas
- ✓ Various data models
- ✓ Horizontal scaling
- ✓ High throughput
- ✓ Eventual consistency

Best for: Unstructured data, real-time apps, massive scale

🎓 **This course:** We focus on SQL (RDS) because analytics workflows need complex queries, joins, and consistent data

Two Database Workloads

Understanding what your database will DO

OLTP
Online Transaction Processing

- Many small, fast operations
- INSERT, UPDATE, DELETE
- Low latency required
- Current, detailed data
- Normalized schemas (3NF)

Examples: Orders, banking, bookings

OLAP
Online Analytical Processing

- Complex analytical queries
- SELECT with aggregations
- Throughput over latency
- Historical, aggregated data
- Denormalized (star/snowflake)

Examples: Reports, trends, dashboards

This course: We focus on OLAP patterns — analyzing business data with complex queries and aggregations

OLTP vs OLAP: The Full Picture

Aspect	OLTP	OLAP
Purpose	Run the business	Analyze the business
Users	Customers, clerks	Analysts, managers
Queries	Simple, predefined	Complex, ad-hoc
Data	Current, detailed	Historical, aggregated
Volume	Thousands of small txns/sec	Few complex queries/hour
Schema	Normalized (3NF)	Denormalized (star/snowflake)
Optimized for	Write speed	Read speed
AWS Service	RDS, Aurora	Redshift, Athena

OLTP Example
"Insert order #12345 for customer John, 2 items, total $50"

OLAP Example
"Show total sales by region and product category for Q4 2025"

Modern trend: HTAP (Hybrid) databases handle both workloads. PostgreSQL can do both reasonably well!

NoSQL Database Types

Four data models — each optimized for specific access patterns

Document Store

Stores **JSON/BSON documents** with flexible, nested structures. Each document is self-contained — no rigid schema required.

MongoDB, CouchDB, aws DocumentDB

Content management, user profiles, catalogs

Key-Value Store

The simplest NoSQL model: **key → value** pairs. Blazing fast lookups by key, but no querying by value. A giant distributed hash map.

Redis, Memcached, aws DynamoDB

Caching, sessions, shopping carts, leaderboards

Column-Family Store

Data organized in **columns grouped by access pattern**, not rows. Optimized for massive writes and reading specific column families.

Cassandra, HBase, aws Keyspaces

Time-series, IoT telemetry, event logging

Graph Database

Data as **nodes (entities) and edges (relationships)**. Excels at traversing connections — friends of friends, shortest paths, patterns.

Neo4j, JanusGraph, aws Neptune

Social networks, fraud detection, recommendations

Key insight: Choose based on your **query patterns**, not just data structure.
Document for flexibility, Key-Value for speed, Column for scale, Graph for relationships.

Database Products Landscape

SQL Databases

PostgreSQL
Open source, feature-rich, extensible
Our Choice

MySQL / MariaDB
Popular, web apps, WordPress

SQL Server
Enterprise, .NET ecosystem

Oracle Database
Enterprise, legacy systems

SQLite
Embedded, mobile apps, testing

NoSQL Databases

MongoDB
Document store, JSON, flexible schema

Redis
In-memory, caching, real-time

Cassandra
Wide-column, distributed, Netflix

Neo4j
Graph database, relationships

Elasticsearch
Search engine, full-text, logging

Cloud Database Services (DBaaS)

Database as a Service — Managed databases in the cloud

Cloud	SQL Service	NoSQL Service	Key Features
aws AWS	RDS, Aurora	DynamoDB, DocumentDB	Largest ecosystem
Azure	SQL Database, PostgreSQL	Cosmos DB	Microsoft integration
G GCP	Cloud SQL, Spanner	Firestore, Bigtable	Data analytics focus

aws

AWS RDS

PostgreSQL, MySQL, MariaDB, Oracle, SQL Server

Our Focus

Azure SQL

SQL Server managed, Cosmos DB multi-model

Cloud SQL

PostgreSQL, MySQL, BigQuery analytics

Why managed? No patching, automated backups, easy scaling, high availability — you focus on data, not infrastructure

10-Minute Break

Stretch, grab coffee, check messages

Back at **10:15** sharp

Next: Relational model fundamentals and schema design

Part 3: Relational Model & SQL

Schema design, normalization, keys, and CRUD operations

What is a Relational Database?

Core Concepts

Table (Relation)
Collection of related data in rows and columns

Column (Attribute)
A field with a name and data type

Row (Tuple)
A single record in the table

Relationship
Connection between tables via keys

Think of a Spreadsheet

customers table

id	name	email	city
1	Ana Garcia	ana@email.com	Barcelona
2	John Smith	john@email.com	London
3	Marie Dupont	marie@email.com	Paris

id = Primary Key (unique identifier)

Key difference from spreadsheets: Enforced data types, constraints, relationships between tables

Schema Normalization

Organizing data to reduce redundancy and improve integrity

Bad Design (Denormalized)

orders table

order_id	customer_name	customer_email	product	qty
1	Ana Garcia	ana@email.com	Laptop	1
2	Ana Garcia	ana@email.com	Mouse	2
3	John Smith	john@email.com	Laptop	1

⚠ Problems: - Customer data repeated - Update one email = update many rows - Inconsistency risk

Good Design (Normalized)

customers | orders

id	name	email
1	Ana Garcia	ana@...
2	John Smith	john@...

id	customer_id	product	qty
1	1	Laptop	1
2	1	Mouse	2
3	2	Laptop	1

✓ Benefits: - No redundancy - Update once, affects all - Data integrity

🎓 **Normalization goal:** Each piece of data stored in exactly ONE place. Relationships connect the pieces.

Keys: The Glue of Relational Databases

Primary Key (PK)

Uniquely identifies each row in a table

- ✓ Must be **unique**
- ✓ Cannot be **NULL**
- ✓ Should be **immutable**
- ✓ Often auto-incremented integer

```
CREATE TABLE customers (
  id SERIAL PRIMARY KEY,  -- Auto-increme
  name VARCHAR(100),
  email VARCHAR(255) UNIQUE
);
```

Foreign Key (FK)

References a primary key in another table

- ✓ Creates **relationships**
- ✓ Enforces **referential integrity**
- ✓ Can be **NULL** (optional)
- ✓ Prevents orphan records

```
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  customer_id INT REFERENCES customers(id
  product VARCHAR(100),
  quantity INT
);
```

Relationships Between Tables

One-to-One

One customer has one profile

Example:
`users` - `user_profiles`

One-to-Many

One customer has many orders

Example:
`customers` → `orders`

Most Common

Many-to-Many

Students enroll in many courses

Example:
`students` - `courses`
(via `enrollments`)

Many-to-Many requires a junction table: `enrollments(student_id, course_id)` — each row represents one enrollment

SQL: Structured Query Language

What is SQL?

The universal language for relational databases

- ✓ **Declarative** — Say WHAT, not HOW
- ✓ **Standard** — Works across databases
- ✓ **Powerful** — Complex queries in one statement
- ✓ **50+ years old** — Still the standard

SQL Categories

Category	Commands	Purpose
DDL	CREATE, ALTER, DROP	Define structure
DML	SELECT, INSERT, UPDATE, DELETE	Manipulate data
DCL	GRANT, REVOKE	Control access

CRUD Operations

The 4 fundamental data operations

- **C** — **Create**
  INSERT INTO ...
- **R** — **Read**
  SELECT ... FROM ...
- **U** — **Update**
  UPDATE ... SET ...
- **D** — **Delete**
  DELETE FROM ...

CREATE: Defining Tables

CREATE TABLE Syntax

```
CREATE TABLE customers (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  city VARCHAR(100),
  created_at TIMESTAMP DEFAULT NOW()
);
```

Element	Meaning
`SERIAL`	Auto-incrementing integer
`VARCHAR(n)`	Variable-length string
`NOT NULL`	Required field
`UNIQUE`	No duplicates
`DEFAULT`	Value if not provided

Common Data Types

Type	PostgreSQL	Example
Integer	`INT`, `SERIAL`	id, quantity
Decimal	`NUMERIC(10,2)`	price, amount
Text	`VARCHAR(n)`, `TEXT`	name, email
Boolean	`BOOLEAN`	is_active
Date/Time	`DATE`, `TIMESTAMP`	created_at
JSON	`JSONB`	metadata

PostgreSQL bonus: Supports `JSONB` for semi-structured data — best of both SQL and NoSQL worlds

INSERT, SELECT, UPDATE, DELETE

INSERT (Create)

```
INSERT INTO customers (name, email, city)
VALUES ('Ana Garcia', 'ana@email.com', 'Barcelona');

-- Multiple rows
INSERT INTO customers (name, email, city)
VALUES
  ('John Smith', 'john@email.com', 'London'),
  ('Marie Dupont', 'marie@email.com', 'Paris');
```

SELECT (Read)

```
-- All columns
SELECT * FROM customers;

-- With filter and sort
SELECT name, email FROM customers
WHERE city = 'Barcelona'
ORDER BY name;
```

UPDATE

```
UPDATE customers
SET city = 'Madrid'
WHERE id = 1;
```

DELETE

```
DELETE FROM customers
WHERE id = 1;
```

ALWAYS use WHERE!
Without WHERE, affects ALL rows

AWS RDS: Relational Database Service

What is RDS?

Fully managed relational database service

- ✓ **6 engines:** PostgreSQL, MySQL, MariaDB, Oracle, SQL Server, Aurora
- ✓ **Automated:** Backups, patches, scaling
- ✓ **High availability:** Multi-AZ deployment
- ✓ **Security:** VPC, encryption, IAM

Free Tier

db.t3.micro
1 vCPU, 1GB RAM

750 hours/month
12 months free

20 GB storage
General Purpose SSD

20 GB backups
Automated snapshots

You Manage

- Schema design & queries
- Performance tuning & indexes
- Access control (users, grants)

AWS Manages

- Hardware provisioning
- Database software patches
- Backups & recovery

⚡ **AWS Aurora:** MySQL/PostgreSQL-compatible engine with 5x throughput, automatic scaling, and serverless option — ideal for production workloads

Hands-On: RDS PostgreSQL with CloudFormation

1. **Deploy CloudFormation Stack**
   VPC + RDS PostgreSQL + EC2 Jupyter

2. **Access JupyterLab**
   http://<EC2-IP>:8888 with token

3. **Run SQL Practice Notebook**
   sql_practice.ipynb with %%sql magic

SQL Exercises

- ✓ SELECT basics & column selection
- ✓ WHERE clause & filtering
- ✓ ORDER BY sorting
- ✓ Aggregate functions (COUNT, AVG)
- ✓ UPDATE & LIKE patterns

Practice Repository

github.com/oriolrius/rds-pg

- CloudFormation template
- Jupyter notebooks
- Full VPC architecture

What Gets Deployed

VPC	10.0.0.0/16 with 4 subnets
RDS	PostgreSQL 15, db.t3.micro
EC2	JupyterLab + Python 3.13

Security groups restrict access to your IP only

Open Repository

45 min hands-on

Key Takeaways

Three Storage Paradigms
Filesystem (EFS), Object Storage (S3), Databases (RDS). Choose based on access patterns.

CRUD with Care
CREATE, READ, UPDATE, DELETE. Always use WHERE on UPDATE/DELETE. Test on small data first.

SQL for Analytics
Relational databases excel at complex queries, joins, and consistent data. NoSQL for specific use cases.

RDS = Managed PostgreSQL
No patching, automated backups, easy scaling. You focus on SQL, AWS handles the rest.

Keys Are Essential
Primary keys identify rows uniquely. Foreign keys create relationships. Both enforce data integrity.

PostgreSQL FTW
Most advanced open-source DB. JSON support, extensions, industry standard for analytics.

What We Built Today

AWS RDS Instance

PostgreSQL 16 running in eu-west-1 with public access

Normalized Schema

Customers, Products, Orders, Order_Items — properly related

CRUD Skills

INSERT, SELECT, UPDATE, DELETE with real business data

Achievement Unlocked

You now have a live cloud database with structured business data!

Common Pitfalls to Avoid

UPDATE without WHERE
Updates ALL rows! Always test with SELECT first.

DELETE without WHERE
Deletes ALL rows! Same rule: SELECT first.

Forgetting Foreign Keys
Orphan records corrupt your data. Use REFERENCES.

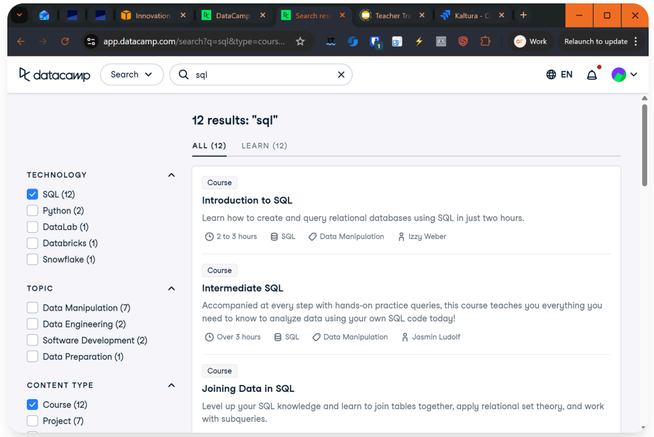
Public Database Access
OK for learning, never in production. Use VPCs.

Not Saving SQL Scripts
Keep your CREATE and INSERT statements versioned.

$ RDS Running 24/7
~$12/month for db.t3.micro. Stop when not practicing.

Level Up Your SQL Skills!

Want to practice more? DataCamp offers excellent interactive SQL courses



Introduction to SQL | **Intermediate SQL** | **Joining Data in SQL**

Explore DataCamp SQL Courses

Before Next Session

Homework

1. **Keep RDS Running (or Stop)**
   We need it for Session 3. Stop if not practicing.
2. **Practice CRUD Operations**
   Add more customers, products, orders. Try different SELECTs.
3. **Read About JOINs**
   Preview: INNER, LEFT, RIGHT, FULL joins
4. **Save Your SQL Scripts**
   Git repo or local folder — you'll need them!

Next Session Preview

Session 3: Advanced SQL

- ✓ JOIN operations (combine tables)
- ✓ Aggregations (SUM, COUNT, GROUP BY)
- ✓ Subqueries and CTEs
- ✓ Triggers for automation
- ✓ Views for reusable queries

Wednesday, Feb 18 | 9:00-12:00

Bring: Your RDS connection working, SQL scripts saved

Questions?

Now is the time to ask anything about:

- Storage types
- SQL & schemas
- AWS RDS
- Final project

Office hours: **Wednesday 14:30-16:00**

Or email for appointment (not questions already in syllabus)

Thank You!

See you next class!

Questions? → joseporiol.rius@esade.edu